PROBLEME DES DATEIANSATZES

1. **Redundanz und Inkonsistenz** (Kopien widersprechen sich) · 2. **Programm-Daten-Abhängigkeit** · 3. **fehlender Datenaustausch**, wenig Flexibilität · 4. **mangelnde Datensicherheit**

DB, DBMS, DATA DICTIONARY

- **Datenbank (DB):** Sammlung logisch zusammenhängender Daten für einen Zweck.
- **DBMS:** Software, mit der man eine DB erstellt und pflegt.
- **Datenbanksystem** = DB + DBMS (+ Anwendungen).
- **Data Dictionary:** Datenbankkatalog mit **Metadaten** („Daten über Daten"): Tabellen, Spalten, Datentypen, Einschränkungen. Daneben die Nutzdaten.
- **Vorteile:** Redundanzkontrolle (Views), Zugriffsbeschränkungen, dauerhafte Speicherung, Mehrbenutzerzugriff, Integritätsbedingungen, Wiederherstellung.
- **Nachteile:** hohe Anfangskosten, für einfache Fälle zu komplex, Mehraufwand für Sicherheit/Wiederherstellung.

ER-MODELL (KONZEPTUELLE EBENE)

Ebenen: konzeptuell (ER) → logisch (Tabellen) → physisch (Dateien, Indizes).

- **Entität:** ein Ding, über das Daten gespeichert werden. **Entitätstyp** (alle gleichartigen) = Rechteck.
- **Attribut** = Ellipse, **Schlüsselattribut** = Ellipse unterstrichen. **Beziehungstyp** = Raute, kann eigene Attribute haben (Rückgabedatum einer Ausleihe).

Kardinalität: 1:1 (Leser – Bibliothekskarte) · 1:n (Rechnung – Posten) · m:n (Student – Vorlesung).

1. Wie viele B hat ein A?
2. Wie viele A hat ein B?
3. Zusammensetzen. Zweimal „viele" → m:n.

RELATIONALES MODELL

- **Relation** = Tabelle, **Tupel** = Zeile, **Attribut** = Spalte mit Datentyp; Werte atomar.
- **Primärschlüssel:** identifiziert jede Zeile eindeutig. **Fremdschlüssel:** verweist auf den Primärschlüssel einer anderen Tabelle.

ER-Element	wird zu
Entitätstyp	Tabelle, Schlüssel → Primärschlüssel
1:n	Fremdschlüssel auf der **n-Seite**
1:1	Fremdschlüssel auf einer Seite
m:n	Hilfstabelle mit beiden Fremdschlüsseln (+ Beziehungsattribute)

m:n Beispiel: STUDENT(MatrNr, Name), VORLESUNG(VNr, Titel), BESUCHT(MatrNr, VNr).

SQL ÄNDERN (DEKLARATIV: WAS, NICHT WIE)

```
CREATE TABLE Firma (Kurzname char(15) not null, Name
char(40));
INSERT INTO Student (MatrNr, Semester) VALUES
(12345678, 4);
UPDATE Student SET Exmatrikuliert = TRUE WHERE
Semester > 99;
DELETE FROM Student WHERE MatrNr = 87654321;
```

SQL LESEN

Ausführungsreihenfolge: 1. **FROM** (Tabellen, Join) → 2. **WHERE** (Zeile für Zeile) → 3. **GROUP BY** → 4. **SELECT**.

- **Aggregate:** SUM, MAX, MIN, AVG, COUNT. Ohne GROUP BY genau eine Zeile, mit GROUP BY eine je Gruppe.
- **Join:** alle Tabellen ins FROM, „Fremdschlüssel = Primärschlüssel" ins WHERE.

```
SELECT PNUMMER, ABTNR, NNAME
FROM PROJEKT, ABTEILUNG, ANGESTELLTER
WHERE ABTNR = ABTNUMMER AND MGRSSN = SSN
  AND PSTANDORT = 'Stafford';
```

Projekte 10, 30 → Abteilung 4 → MGRSSN 987654321 → Wallace: **10 | 4 | Wallace**, **30 | 4 | Wallace**.

GROUP BY ABT mit AVG(GEHALT): Abt. 4 = (25000 + 43000 + 25000) / 3 = **31000**.

SQL SCHREIBEN

1. Was ausgeben? → SELECT; „wie viele/Summe/Durchschnitt/höchstes" → Aggregat.
2. Wo stehen die Spalten? → FROM, bei mehreren Tabellen Join ins WHERE.
3. Welche Zeilen? → WHERE mit AND/OR, Texte in '…'.
4. „pro"/„je" → GROUP BY. „Projekte je Abteilung": `SELECT ABTNR, COUNT(*) FROM PROJEKT GROUP BY ABTNR;`

OLTP UND OLAP

- **OLTP:** Tagesgeschäft (Bestellung, Buchung), aktuelle Daten, ganze Datensätze lesen/schreiben, Row Store (zeilenweise); Oracle, MySQL.
- **OLAP:** Analyse (BI, Data Warehouse), mit Historie, fast nur lesen, Aggregate, Column Store (spaltenweise); Amazon Redshift, SAP HANA.

ETL (Extract, Transform, Load) bringt Daten von OLTP nach OLAP.

TYPISCHE FEHLER

- Redundanz allein als Problem nennen; das Problem ist die Inkonsistenz.
- DBMS (Software) und Datenbank (Datenbestand) vertauschen.
- Kardinalität nur aus einer Richtung fragen.
- Fremdschlüssel bei 1:n auf die 1-Seite setzen.
- UPDATE/DELETE ohne WHERE → alle Zeilen betroffen.
- Typografische statt gerader Hochkommas; DELETE ohne FROM.
- `>` und `>=` verwechseln (genau 30000 fällt bei `> 30000` heraus); AND und OR verwechseln.
- Aggregat ohne GROUP BY, aber mehrere Zeilen erwarten.
- Join-Bedingung vergessen → jede Zeile mit jeder kombiniert.
- Data Warehouse als OLTP-System einordnen.